



**Samueli**  
School of Engineering

---

# CS-M151B

# Computer Systems Architecture

---

Blaise Tine  
UCLA Computer Science

# Recap: Control Hazards

---

CPU must answer 3 questions:

- Which type of branch?
- Whether to branch?
- Where to branch?

# Recap: Control Hazards

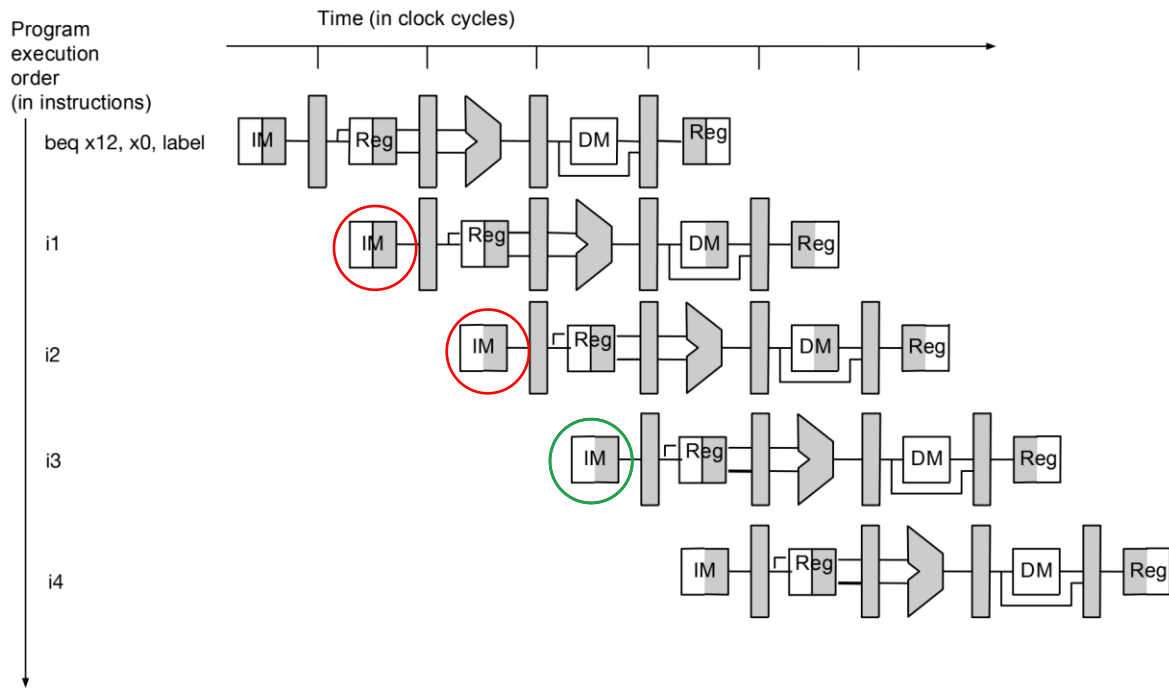
---

## RISC-V branches

- Conditional branches
  - `BEQ r1, r2, imm;    // if (r1 == r2) => PC = PC + imm`
- Unconditional branches
  - `JAL rd, imm;        // rd = PC + 4; PC = PC + imm`
  - `JALR rd, rb, imm;   // rd = PC + 4; PC = rb + imm`

# Recap: Control Hazards

- Conditional: 2 stalls
- Unconditional: 2 stalls
- Branch Target
  - ID/EX
  - EX/MEM
- Branch condition
  - EX/MEM



# Control Hazards Handling

---

- Software Solution: Insert NOPs
  - S/W insert NOPs in delay slots
  - Pros:
    - No H/W cost
  - Cons:
    - Bad performance: still stalling
    - S/W depends on H/W stall count

# Control Hazards Handling

---

- Software Solution: Delayed branch
  - S/W insert useful instructions in delay slots
  - Pros:
    - No H/W cost
    - Improved performance
  - Cons:
    - Hard to find useful instructions
    - S/W depends on H/W stall count

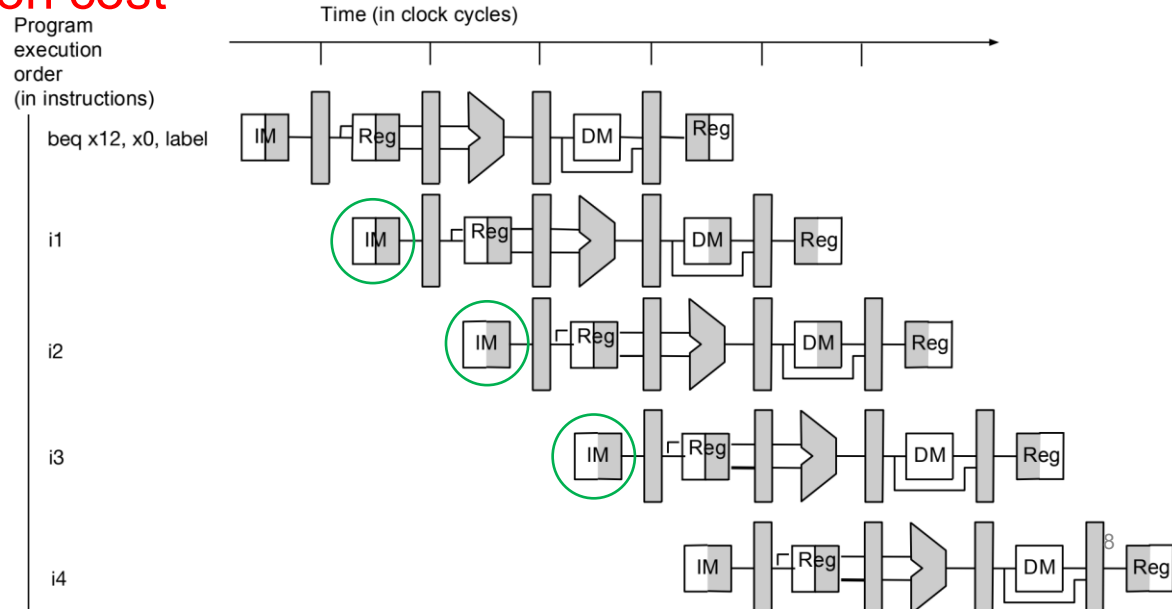
# Control Hazards Handling

---

- Hardware Solution: Speculation
  - Pros:
    - S/W independence
    - Better performance
  - Cons:
    - H/W cost

# Control Hazards: Speculation

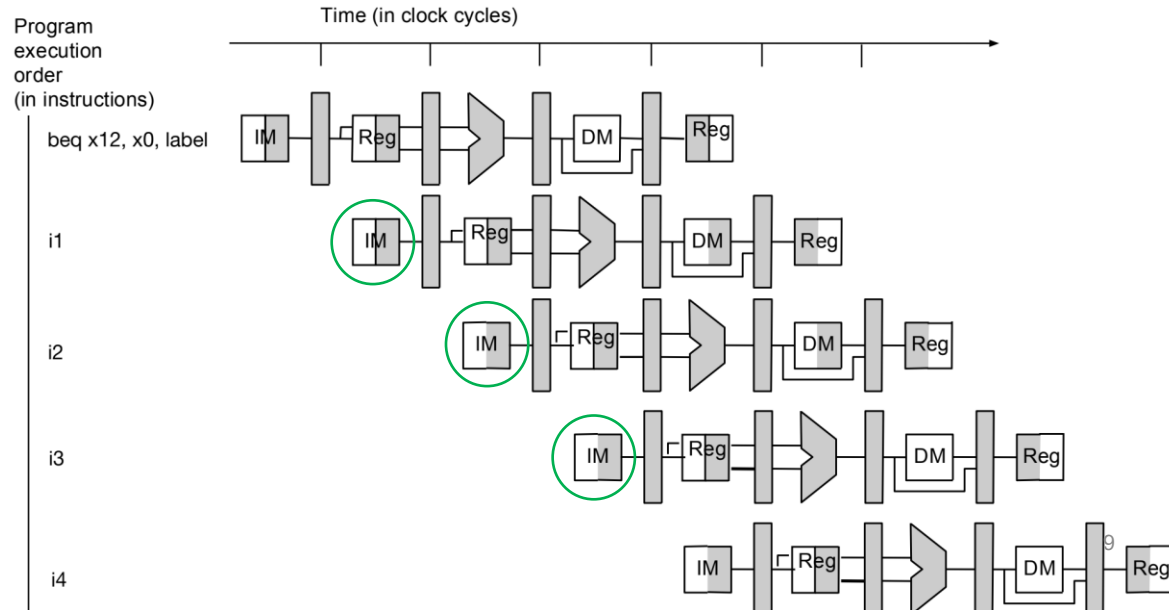
- Predict: Not taken
  - Emulate delay branch
  - Pros: no wasted cycles
  - Cons: mis-prediction cost





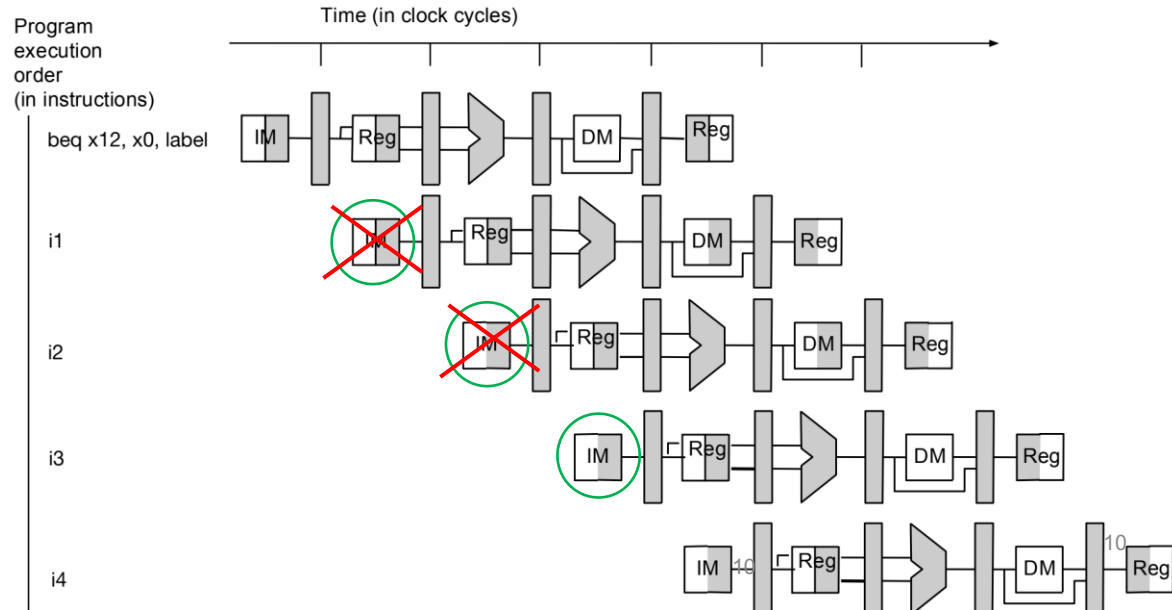
# Control Hazards: Speculation

- Predict: Not taken
  - How to detect misprediction?
    - Branch condition is taken
  - When?
    - MEM stage



# Control Hazards: Speculation

- Predict: Not taken
  - How to recover from misprediction?
    - Flush the pipeline (invalidate each previous stage)
    - Update PC
- Penalty
  - 2 cycles



# Control Hazards: Speculation

---

- Practical Branch stats
  - About 20% of instructions is control-flow
    - 50% of forward branches (if-then-else) are taken
    - 90% of backward branches (loops) are taken
    - 30% of all branches are not-taken

# Control Hazards: Speculation

---

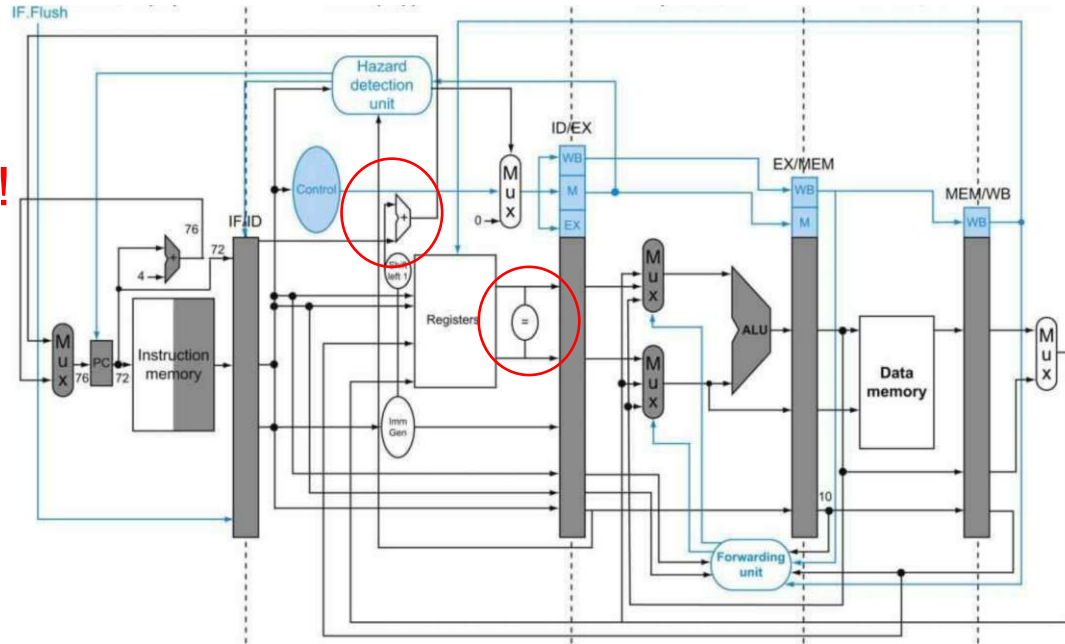
- Practical Branch stats
  - About 20% of instructions is control-flow
    - 50% of forward branches (if-then-else) are taken
    - 90% of backward branches (loops) are taken
    - 30% of all branches are not-taken
- Not-taken prediction accuracy?
  - Only 30%
  - Penalty: 3 cycles

# Whiteboard: Aggressive branch opt

---

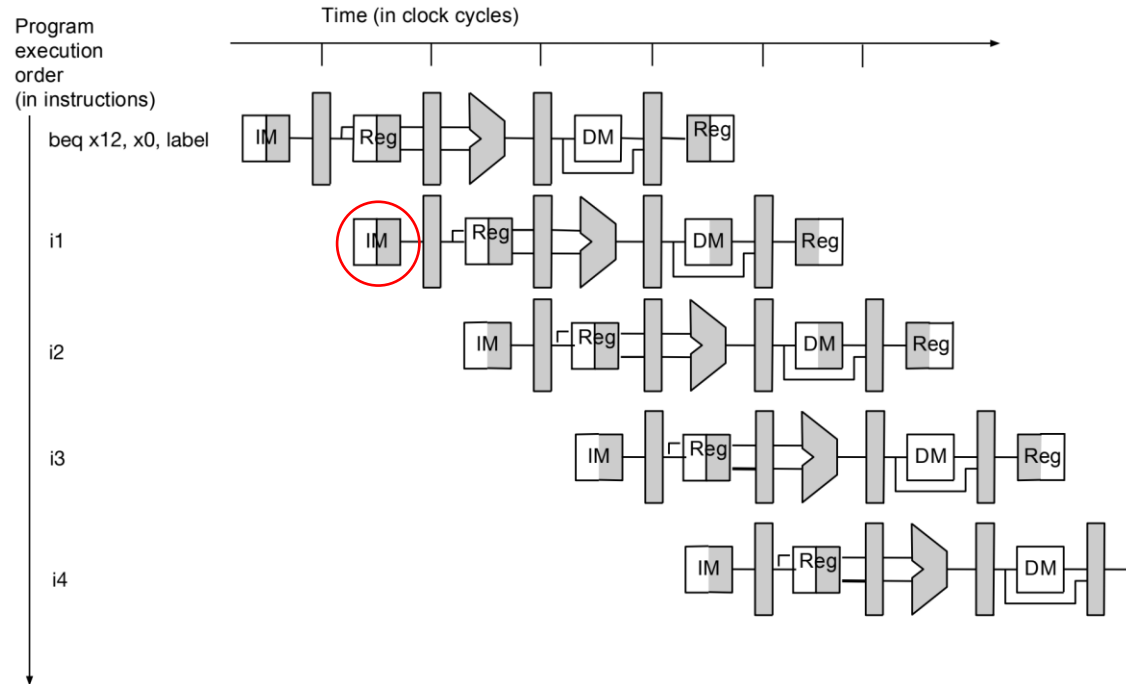
# Control Hazards: Speculation

- Reducing Misprediction Cost
  - More aggressive solution: **Unconditional Branch only**
  - Do NextPC to ID
  - Do PC + imm to ID
- Warning: r1/r2 RAW
  - Need data forwarding!



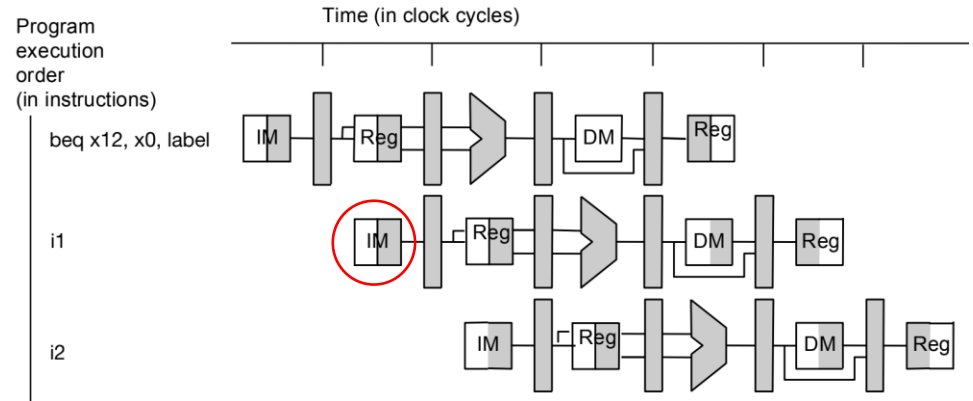
# Control Hazards: Speculation

- Predict: Always Taken
- What do we need?
  - nextPC
  - At fetch stage



# Control Hazards: Speculation

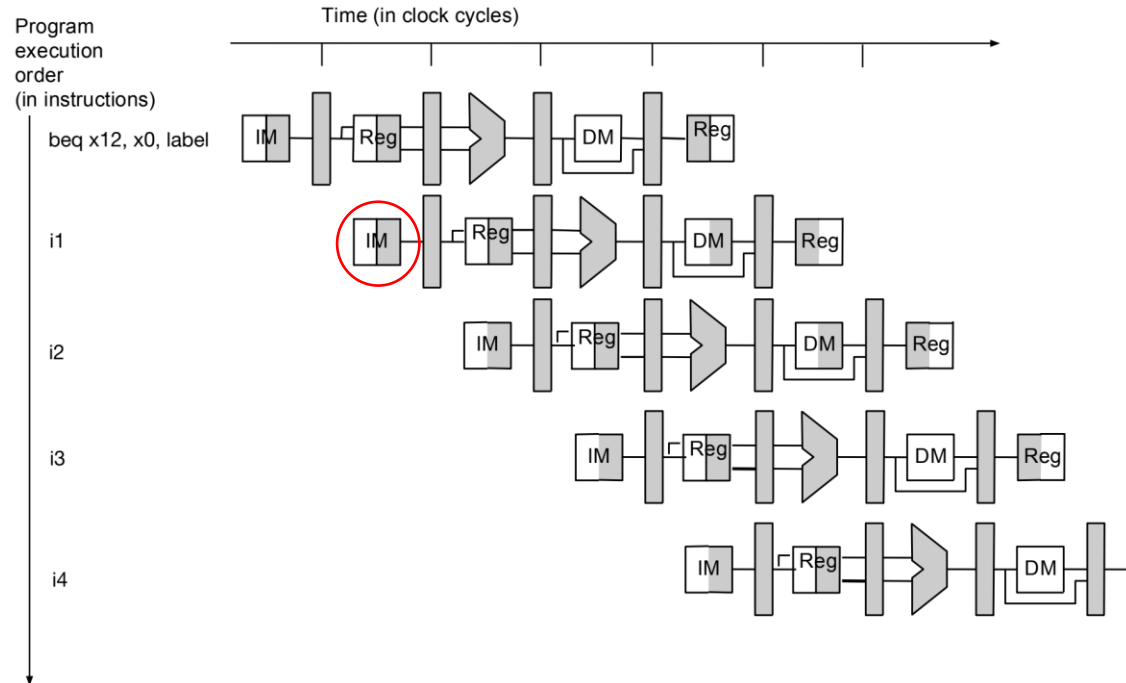
- Predict: Always Taken
  - How to obtain nextPC?
  - We can predict





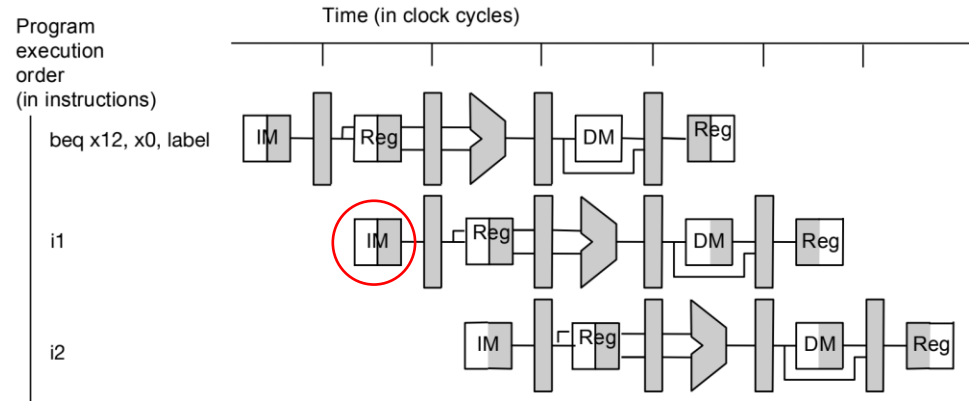
# Control Hazards: Speculation

- Predict: Always Taken
- What do we need?
  - Condition
  - Target



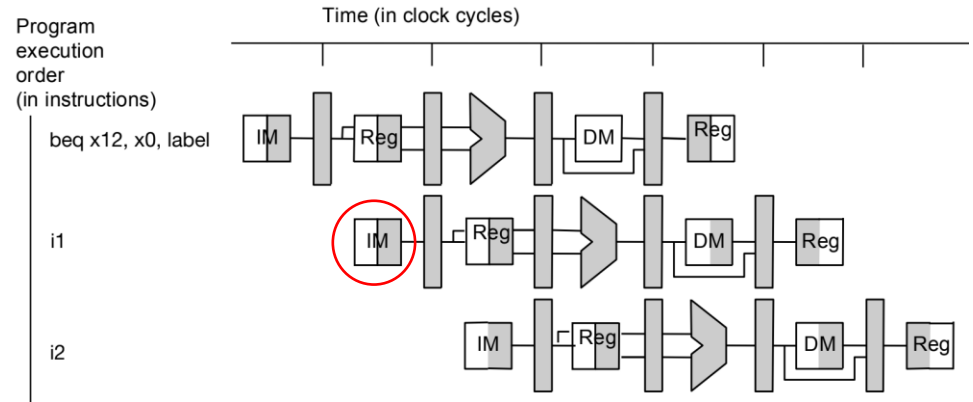
# Control Hazards: Speculation

- Predict: Always Taken
  - How to obtain NextPC?
  - Needed in IF stage
  - We can predict



# Control Hazards: Speculation

- Predict: Always Taken
  - How to obtain the branch target?
  - We can predict



# Control Hazards: Speculation

---

- When do we know the branch target?
  - Direct Jumps, Function calls: JAL
    - $PC = PC + imm$
    - In ID stage
    - Prediction accuracy: High

# Control Hazards: Speculation

---

- When do we know the branch target?
  - Hard to compute conditional branches: BEQ/BNE
    - $PC = (R[rs1] \text{ ops } R[rs2]) ? (PC + imm) : PC + 4$
    - In ID stage
    - Prediction accuracy: High

# Control Hazards: Speculation

---

- When do we know the branch target?
  - Indirect Jumps, Dynamic Calls, Returns: JALR
    - $PC = R[rs1] + imm$
    - After ID stage
    - Prediction accuracy: Low

# Control Hazards: Speculation

---

- How do we predict the address?
  - We need to track the history of previous branch targets
  - If a branch was visited before, reuse its previous target
  - There is a chance it will execute the same path

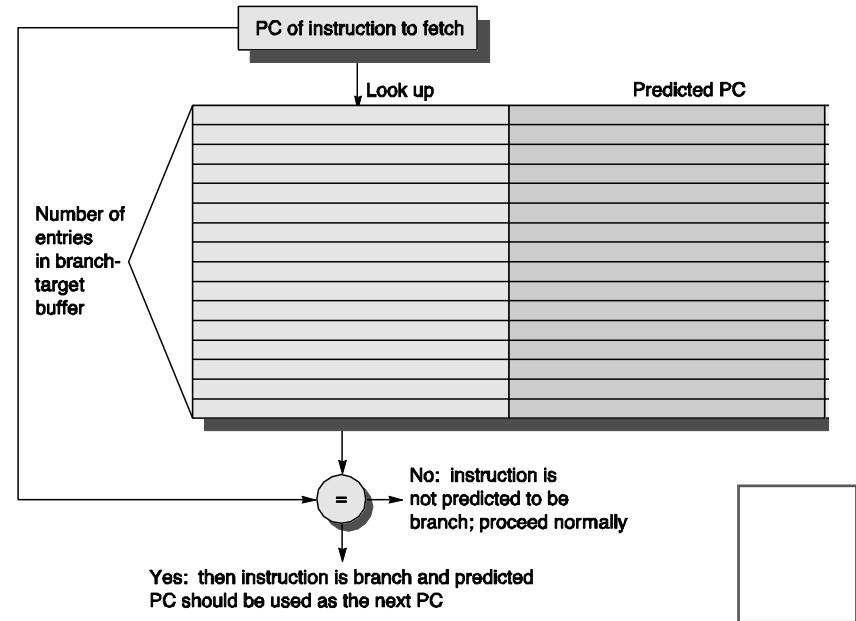
- Branch Target Buffer (BTB)
  - A table that stores branch targets
  - Indexable using the current PC
    - Why using the PC?





# Control Hazards: Speculation

- How BTB works?
  - Use PC to lookup the BTB
  - Found a match?
    - NextPC = BTB entry;
  - Otherwise
    - NextPC = PC + 4;



# Control Hazards: Speculation

---

- BTB implementation questions:
  - Where is BTB accessed?
  - How do we know that the current PC is a branch before accessing the BTB?
  - When do we update the BTB?
  - How do we update the BTB?

# Control Hazards: Speculation

---

- Where is BTB accessed?
  - NextPC is needed in IF stage
  - BTB should be accessed in IF stage

# Control Hazards: Speculation

---

- How do we know that the current PC is a branch before accessing the BTB?
  - Only branches are added into BTB
  - Make sure to clear BTB at startup

# Control Hazards: Speculation

---

- When do we update the BTB?
  - When we can compute branch target
  - In ID stage:
    - Add dedicated adder
    - JALR timing issue – register file access
  - In EX stage:
    - Add dedicated adder
    - No timing issue

# Control Hazards: Speculation

---

- How do we update the BTB?
  - Check if branch
  - Calculate NextPC
  - Update BTB
    - $BTB[PC] = NextPC$

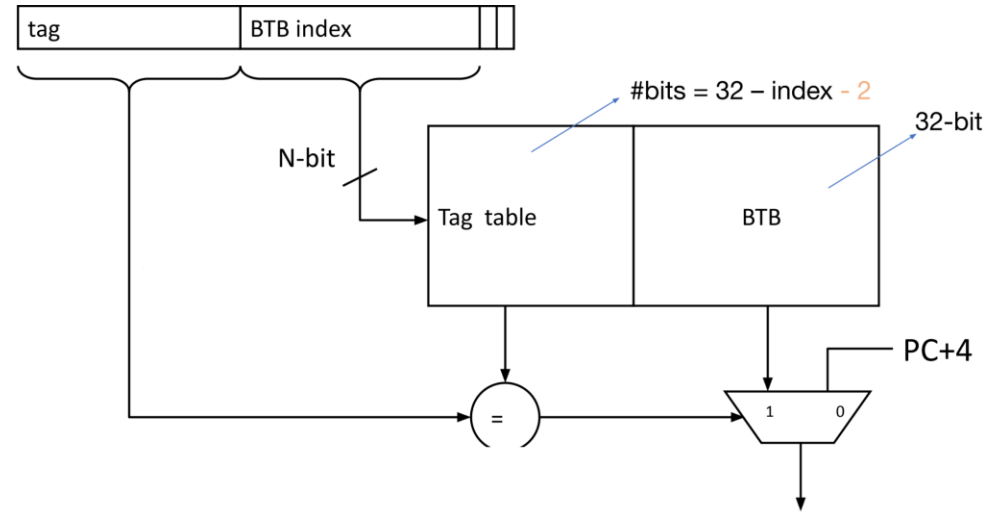
# Control Hazards: Speculation

---

- BTB Hardware Implementation
  - Implementing a hash table in H/W is expensive
    - Lookup area cost
    - Lookup latency
    - Table size must be small

# Control Hazards: Speculation

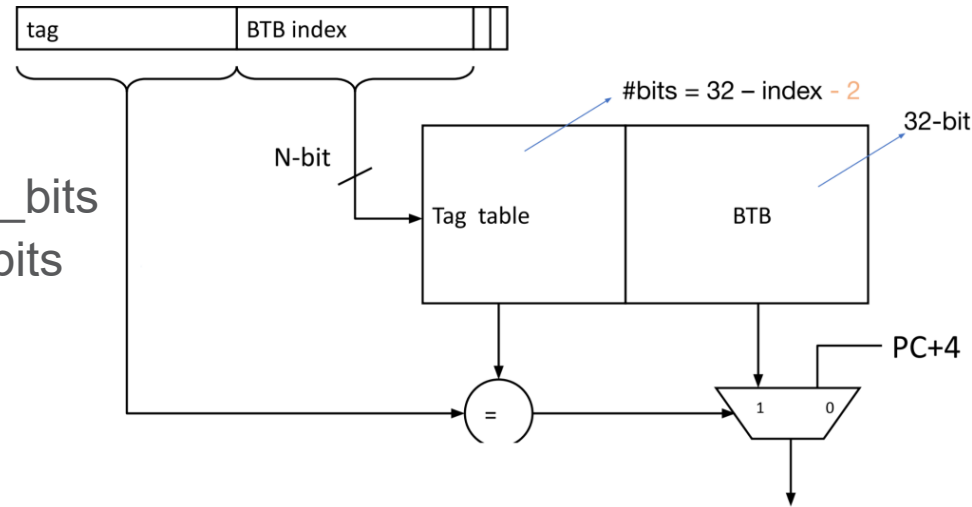
- BTB Hardware Implementation
  - Use a Tagged-based Lookup table
  - Fixed size: `btb_size`
  - Indexing: `btb_index`
  - Lookup: `btb_tag`





# Control Hazards: Speculation

- BTB Hardware Implementation
  - PC address is 4-byte aligned
    - lower 2 bits are zero
  - Formulas:
    - $\text{index\_bits} = \log_2(\text{btb\_size})$
    - $\text{pc\_index} = (\text{pc} \gg 2) \& \text{index\_bits}$
    - $\text{pc\_tag} = (\text{pc} \gg 2) \gg \text{index\_bits}$



# Control Hazards: Speculation

- BTB Hardware Implementation

- Initialization:
  - Clear valid bit at reset
- BTB lookup:
  - `btb <= BTB[pc_index];`
  - `if (btb.valid == 1 && btb.tag == pc_tag) { PC = btb.target; } else { PC = PC + 4; }`
- BTB update:
  - `BTB[pc_index] <= {1, pc_tag, target};`

BTB hardware table

| valid<br>1 bit | tag<br>26 bits | target<br>30 bits | 16 entries<br>↓ |
|----------------|----------------|-------------------|-----------------|
|                |                |                   |                 |
|                |                |                   |                 |
|                |                |                   |                 |

# Attendance

---

- Please Fill the form with today's keyword to register your attendance.



# Control Hazards: Speculation

---

- Improving Branch prediction
  - Predicting always taken is about 70% accurate
  - Better than always not taken (30%).
  - How do we improve the prediction accuracy?

# Control Hazards: Speculation

---

- Code reordering optimization
  - If H/W implements static prediction
    - Not Taken or always Taken
  - Reorder code to prioritize predicted path
  - Pros/cons?
    - Pros: improved accuracy
    - Cons: architecture specific

# Control Hazards: Speculation

---

- BTFNT (Backward Taken, Forward Not Taken)
  - Loops usually have a few iterations, so this is like always predicting that the loop is taken
  - Loop back-edge => backward direction
  - Pros/Cons?
    - Pros: Improved accuracy
    - Cons: limited to loops only

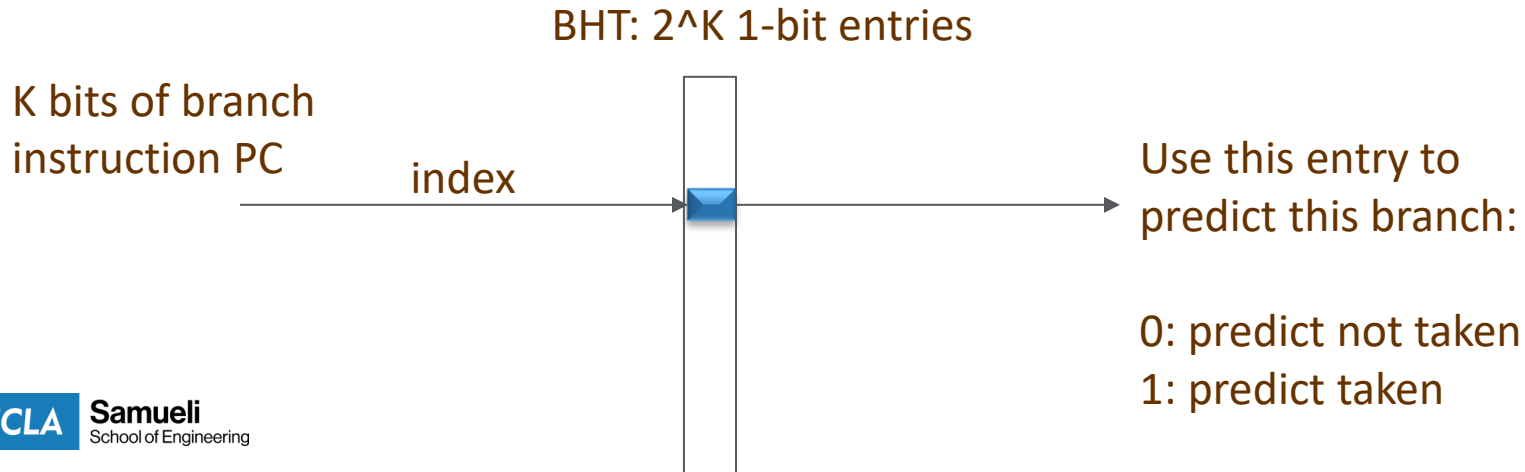
# Control Hazards: Speculation

---

- Using a Branch predictor
  - Predict the outcome dynamically using the branch history
  - Recurring branches tend to repeat their pattern (e.g. loops)

# Control Hazards: Speculation

- One-Bit Branch Predictor
  - BTB entry hold a single bit: taken=1 or not\_taken=0.
  - Index the table using lower K bits of PC
  - Update the table when the condition is resolved.

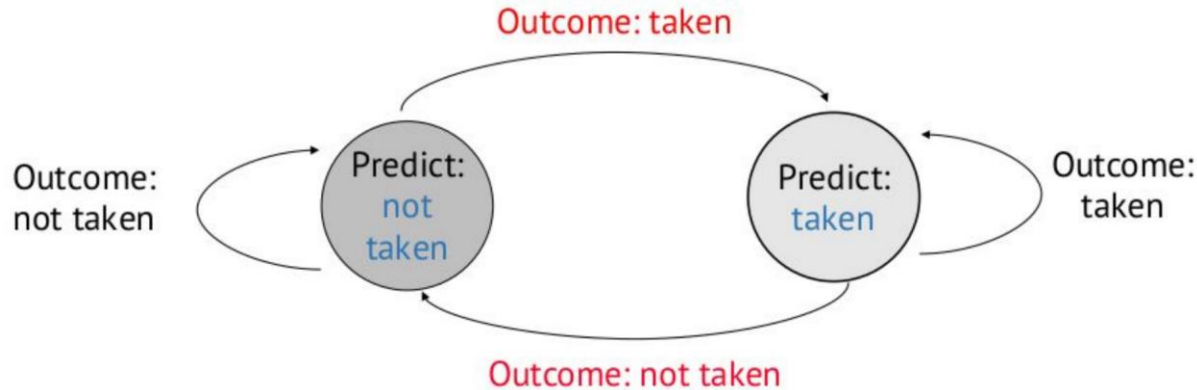




# Control Hazards: Speculation

---

- One-Bit Branch Predictor
  - FSM



# Control Hazards: Speculation

- One-Bit Branch Predictor
  - Examples

DC08: TTTTTTTTTT ... TTTTTTTTTNTTTTTTTTTT...

← 100,000 iterations →

How often is branch outcome != previous outcome?

2 / 100,000

99.9%

DC44: TTTTTT ... TNTTTTTT ... TNTTTTTT ...

← 2 / 100 →

2 / 100

98.0%

DC50: TNTNTNTNTNTNTNTNTNTNTNTNTNTNTNT...

← 2 / 2 →

2 / 2

00.0%

# Control Hazards: Speculation

---

- One-Bit Is not enough!

Example: short loop (8 iterations)

- Taken 7 times, then not taken once
- Act: TTTTTTTNTTTTTTNTTTTTTTNT...
- Pred: XTTTTTTT**T**NTTTTTT**T**NTTTTTTT**T**N
- Corr: Xoooooooo**MM**oooooooo**MM**oooooooo**MM**
- Misprediction rate:  $2/8 = 25\%$

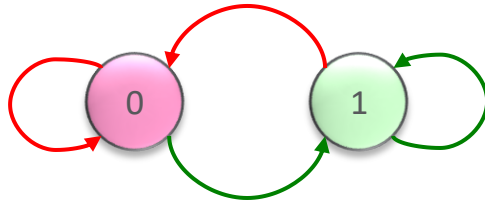
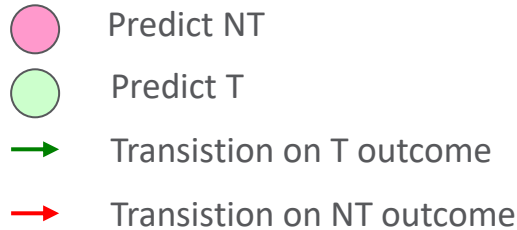
Execute the same loop again

- First always mispredicted (previous outcome was not taken)
- Then 6 predicted correctly
- Then last one mispredicted again

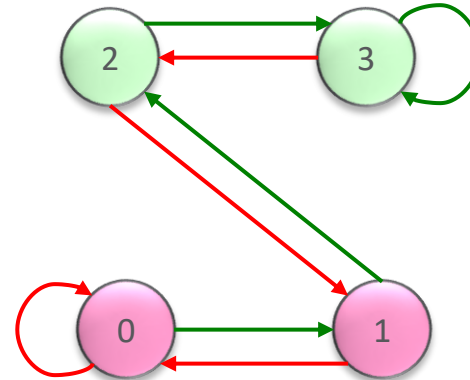
Each fluke/anomaly results in two mispredicts per loop

# Control Hazards: Speculation

- Two-Bit Branch Predictor



FSM for Last-time  
Prediction



FSM for 2bC  
(2-bit Counter)

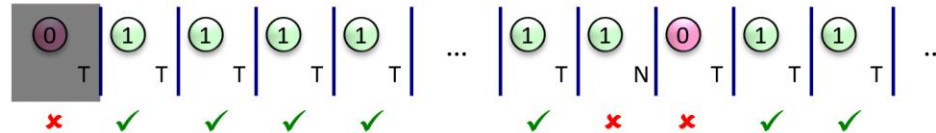
Smith, ISCA 1981

# Control Hazards: Speculation

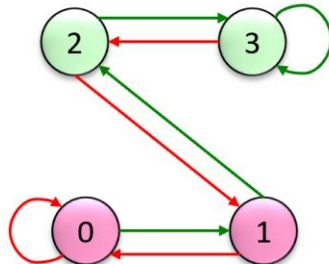
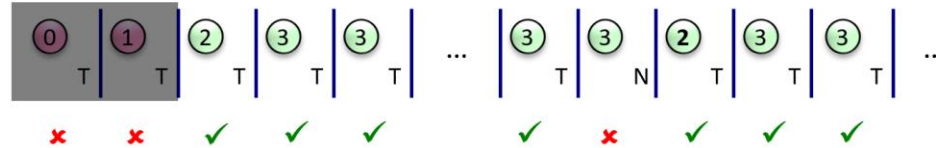
- Two-Bit Branch Predictor

Initial Training/Warm-up

1bC:



2bC:



Only 1 Mispredict per N branches now!

DC08: 99.999%

DC44: 99.0%

DC50: 50.0%

# Control Hazards: Speculation

---

- Can we do better?
  - 1-BC and 2-BC predictors use last branch outcome to predict
  - Observation1: global branch correlation
    - The branch outcome is correlated with outcome from previous branches
  - Observation2: local branch correlation
    - The branch outcome is correlated with prior-outcome from same branch

# Control Hazards: Speculation

---

## Global branch correlation

```
if (cond1) {  
    --/--  
}  
if (cond1 and cond2) {  
    --/--  
}
```

## Local branch correlation

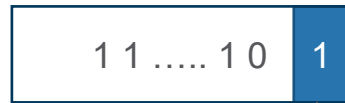
```
for (x: 0 -> 1023) {  
    --/--  
}
```

# Control Hazards: Speculation

## Two-level Branch Predictor

- Level1: history of last N branches
- Level2: last direction the branch took

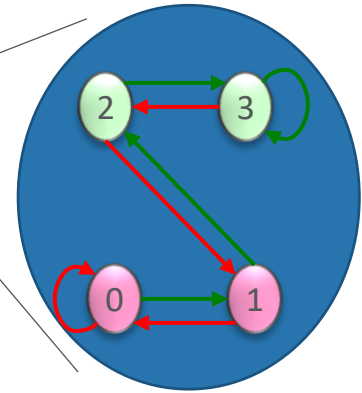
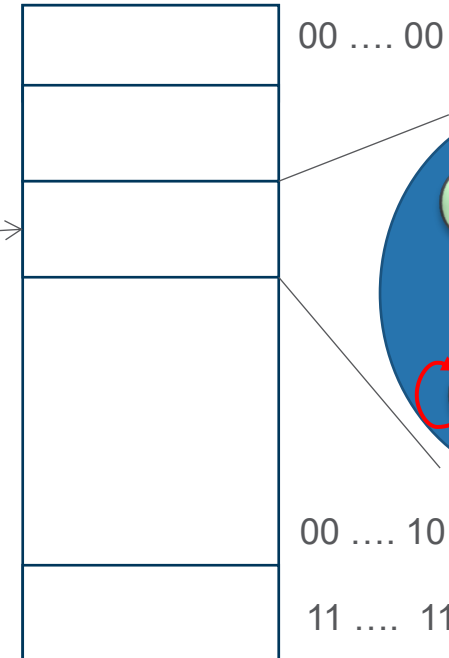
### Branch History Register (BHR)



previous one

index

### 2-Bit BHT



Yeh&patt'92



# Control Hazards: Speculation

## Two-level Branch Predictor

Initialization value (0 or 1)

Past

Present



← History length →

1 : branch is taken  
0: branch is not-taken

New BHR = old BHR<<1 | (br\_dir);

### Example:

|                |             |
|----------------|-------------|
|                | BHR 00000   |
| Br1 : taken    | → BHR 00001 |
| Br2: not-taken | → BHR 00010 |
| Br3: taken     | → BHR 00101 |

# Control Hazards: Speculation

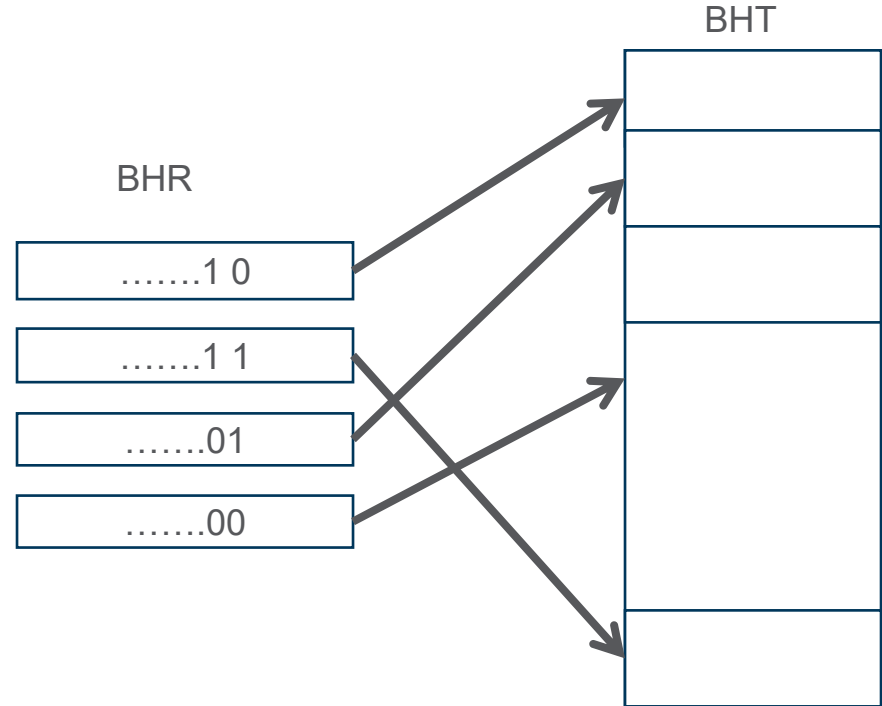
## Two-level Branch Predictor

Branch X: if (cond1)

Branch Y: if (cond 2)

Branch Z : if (cond 1 and cond 2)

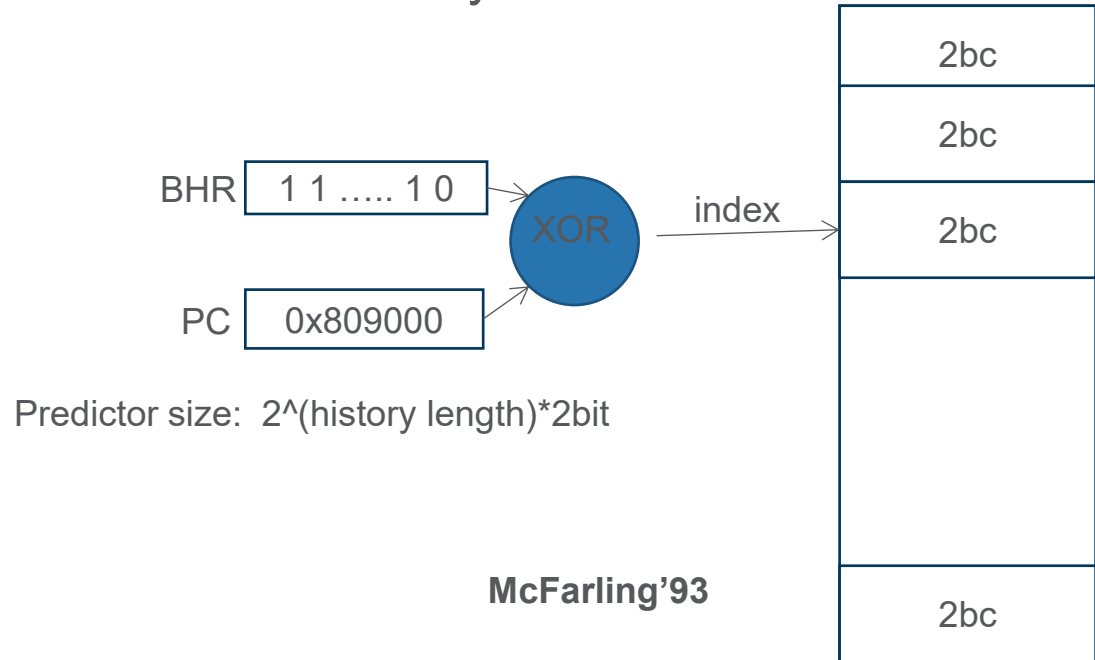
| Branch X | Branch Y | Branch Z |
|----------|----------|----------|
| 1        | 0        | 0        |
| 1        | 1        | 1        |
| 0        | 1        | 0        |
| 0        | 0        | 0        |



# Control Hazards: Speculation

## Gshare Branch Predictor

- Different branches may see the same history but have different outcome
- Need additional context



McFarling'93

# Control Hazards: Speculation

---

## Gshare Branch Predictor: algorithm

```
predict_func(pc, actual_dir) {  
    index = PC xor BHR  
    taken = (2bit_counters[index] >= 2) ? 1 : 0  
    correctly_predicted = (actual_dir == taken) ? 1 : 0 // stats  
}
```

```
update_func(pc, actual_dir) {  
    index = PC xor BHR  
    if (actual_dir)  
        INC_SAT(2bit_counter[index])  
    else  
        DEC_SAT(2bit_counter[index])  
    BHR = BHR << 1 | actual_dir  
}
```

# Control Hazards: Speculation

## Improving Return Call Prediction

- RISC-V JALR
  - $PC = R[rs1] + imm$
- Target depends on rs1

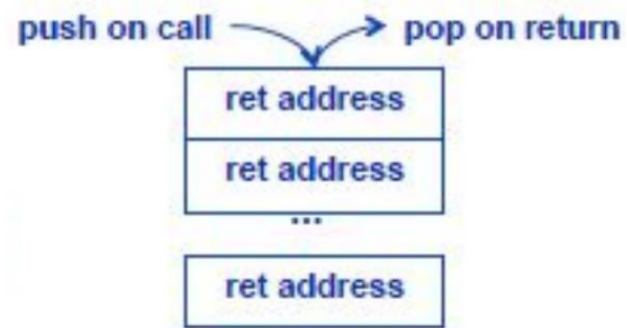
| Control-Flow inst. | Direction at fetch time | Number of possible next fetch addresses? | When is next fetch address resolved? |
|--------------------|-------------------------|------------------------------------------|--------------------------------------|
| Conditional        | Unknown                 | 2                                        | Decode ( $rs1 == rs2$ )              |
| Unconditional      | Always taken            | 1                                        | Decode (PC + offset)                 |
| Call               | Always taken            | 1                                        | Decode (PC + offset)                 |
| Return             | Always taken            | Many                                     | Decode (register dependent)          |
| Indirect           | Always taken            | Many                                     | Decode (register dependent)          |

# Control Hazards: Speculation

---

## Return Stack Buffer (RSB)

- Use hardware stack
  - About 4-8 entries
- On function call:
  - Push current return address
- On function return:
  - Pop last return address
  - Use that for prediction

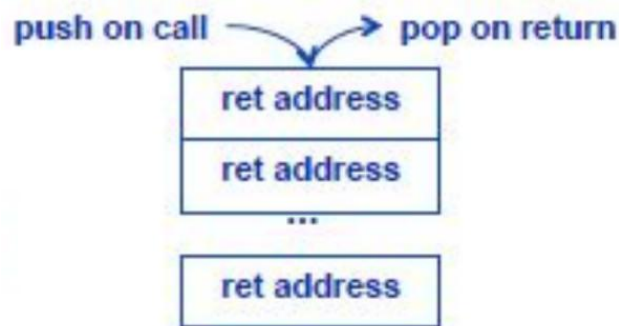


# Control Hazards: Speculation

---

## Return Stack Buffer (RSB)

- Tail recursion optimization
  - if (stack.top == ra) => skip
- Integration with BTB:
  - Predict NextPC from BTB or RSB
  - How to choose?
    - Extend BTB with “isRA” column



# Acknowledgement

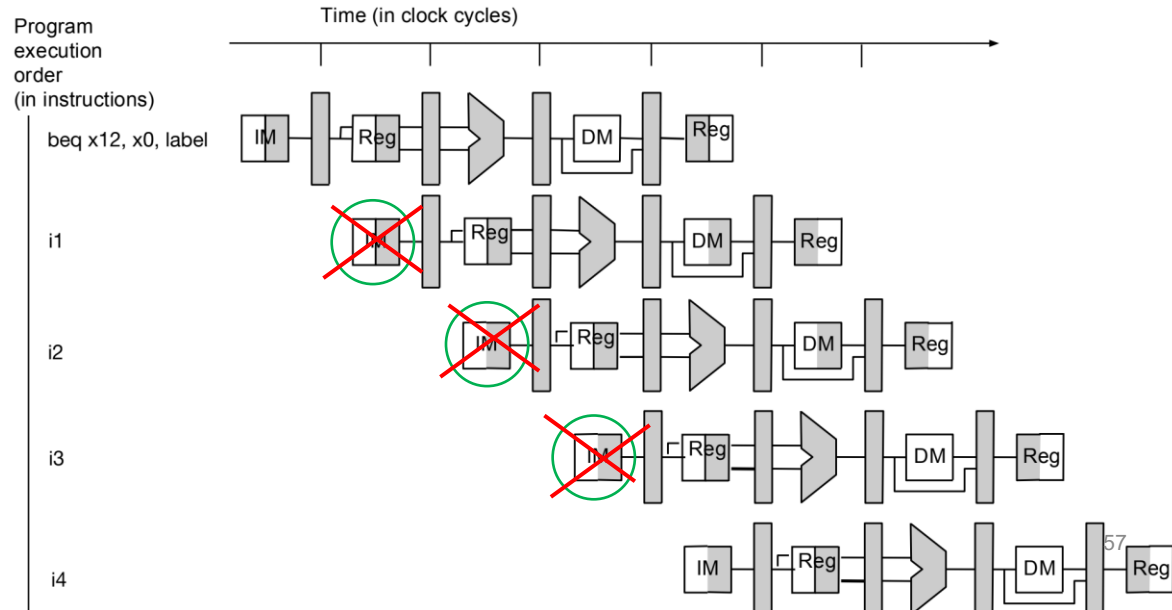
---

- This course is partly inspired and use materials from the following courses created by my colleagues:
  - ECE-M1126C: Nader Sehatbakhsh @ UCLA
  - CS3220: Hyesoon Kim @ gatech



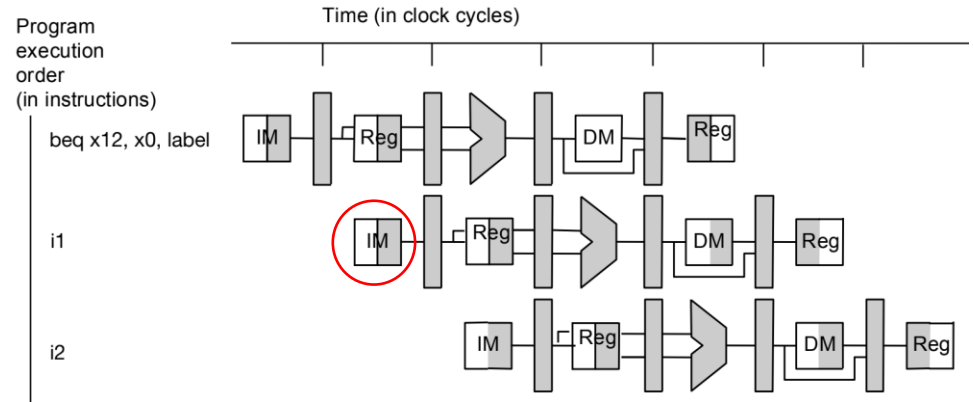
# Control Hazards: Speculation

- Predict: Not taken
  - Branch Target  $\Rightarrow$  PC + 4
  - **Misprediction**  $\Rightarrow$  Flush the pipeline



# Control Hazards: Speculation

- Predict: Always Taken
  - How to obtain NextPC?
  - Needed in IF stage
  - We can predict



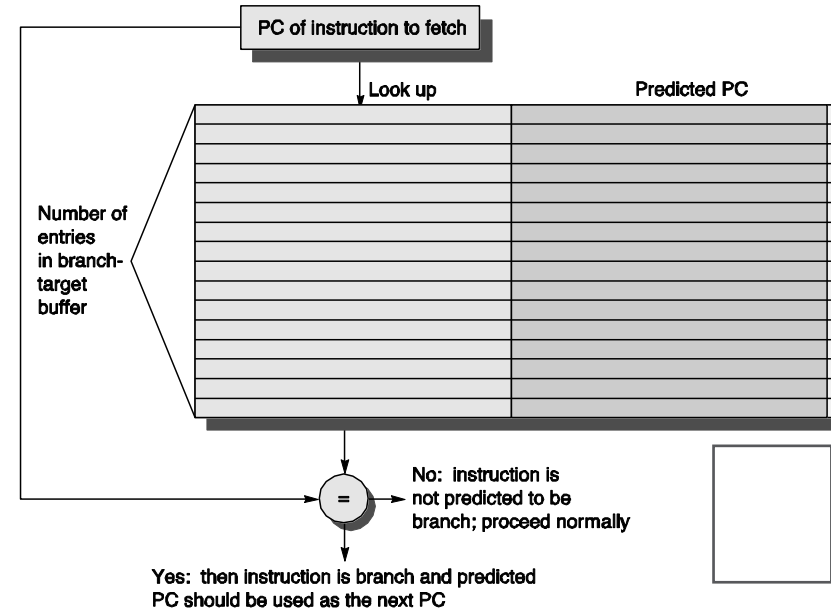
# Control Hazards: Speculation

---

- How do we predict the address?
  - We need to track the history of previous branch targets
  - If a branch was visited before, reuse its previous target
  - There is a chance it will execute the same path

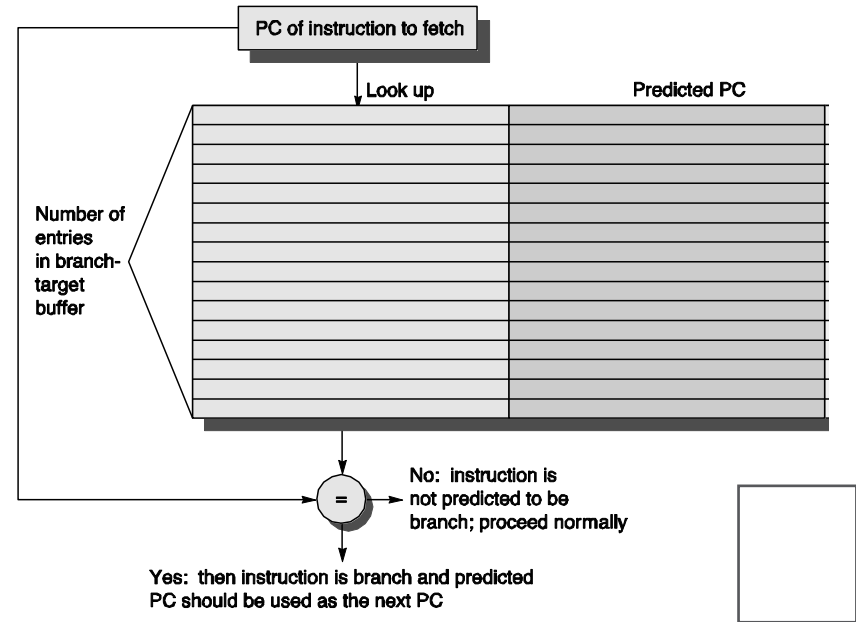
# Control Hazards: Speculation

- Branch Target Buffer (BTB)
  - A table that stores branch targets
  - Indexable using the current PC
  - Why using the PC?



# Control Hazards: Speculation

- How BTB works?
  - Use PC to lookup the BTB
  - Found a match?
    - NextPC = BTB entry;
  - Otherwise
    - NextPC = PC + 4;



# Control Hazards: Speculation

---

- BTB implementation questions:
  - Where is BTB accessed?
  - How do we know that the current PC is a branch before accessing the BTB?
  - When do we update the BTB?
  - How do we update the BTB?

# Control Hazards: Speculation

---

- Where is BTB accessed?
  - NextPC is needed in IF stage
  - BTB should be accessed in IF stage

# Control Hazards: Speculation

---

- How do we know that the current PC is a branch before accessing the BTB?
  - Only branches are added into BTB
  - Make sure to clear BTB at startup



# Control Hazards: Speculation

---

- When do we update the BTB?
  - When we can compute branch target
  - MEM stage: 3 stalls
  - EX stage: 2 stalls
  - ID stage: 1 stall

# Control Hazards: Speculation

---

- How do we update the BTB?
  - Check if branch
  - Calculate NextPC
  - Update BTB
    - $BTB[PC] = NextPC$

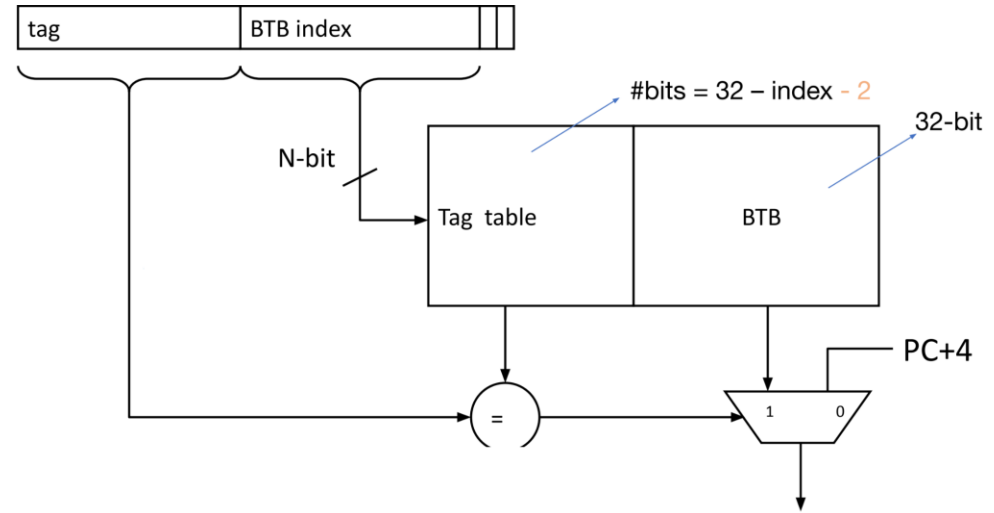
# Control Hazards: Speculation

---

- BTB Hardware Implementation
  - Implementing a hash table in H/W is expensive
    - Lookup area cost
    - Lookup latency
    - Table size must be small

# Control Hazards: Speculation

- BTB Hardware Implementation
  - Use a Tagged-based Lookup table
  - Fixed size: `btb_size`
  - Indexing: `btb_index`
  - Lookup: `btb_tag`

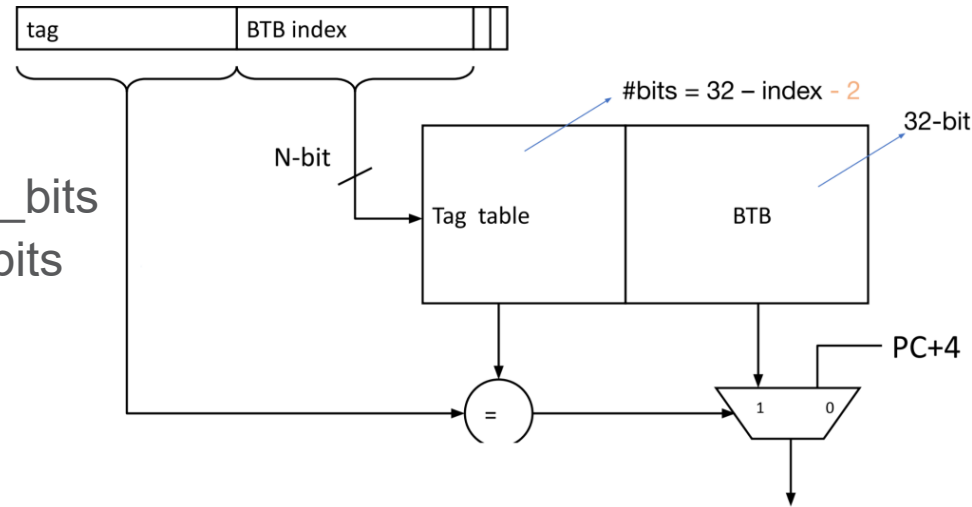


# Control Hazards: Speculation

- BTB Hardware Implementation
  - PC address is 4-byte aligned
    - lower 2 bits are zero
  - Formulas:

tag

    - $\text{index\_bits} = \log_2(\text{btb\_size})$
    - $\text{pc\_index} = (\text{pc} \gg 2) \& \text{index\_bits}$
    - $\text{pc\_tag} = (\text{pc} \gg 2) \gg \text{index\_bits}$



# Control Hazards: Speculation

- BTB Hardware Implementation
  - Initialization:
    - Clear valid bit at reset
  - BTB lookup:
    - `btb <= BTB[pc_index];`
    - `if (btb.valid == 1 && btb.tag == pc_tag) {`  
    `PC = btb.target; } else { PC = PC + 4; }`
  - BTB update:
    - `BTB[pc_index] <= {1, pc_tag, target};`

**BTB hardware table**

| 1 bit | 26 bits | 30 bits |
|-------|---------|---------|
| valid | tag     | target  |
|       |         |         |
|       |         |         |
|       |         |         |
|       |         |         |



16 entries

# Control Hazards: Speculation

---

- Improving Branch prediction
  - Predicting always taken is about 70% accurate
  - Better than always not taken (30%).
  - How do we improve the prediction accuracy?

# Control Hazards: Static Prediction

---

- Code reordering optimization
  - If H/W implements static prediction
    - Not Taken or always Taken
  - Compiler can reorder code to prioritize predicted path
  - Pros/cons?
    - Pros: improved accuracy
    - Cons: architecture specific



# Control Hazards: Static Prediction

---

- BTFNT (Backward Taken, Forward Not Taken)
  - Loops usually have a few iterations, so this is like always predicting that the loop is taken
  - Loop back-edge => backward direction
  - Pros/Cons?
    - Pros: Improved accuracy
    - Cons: limited to loops only

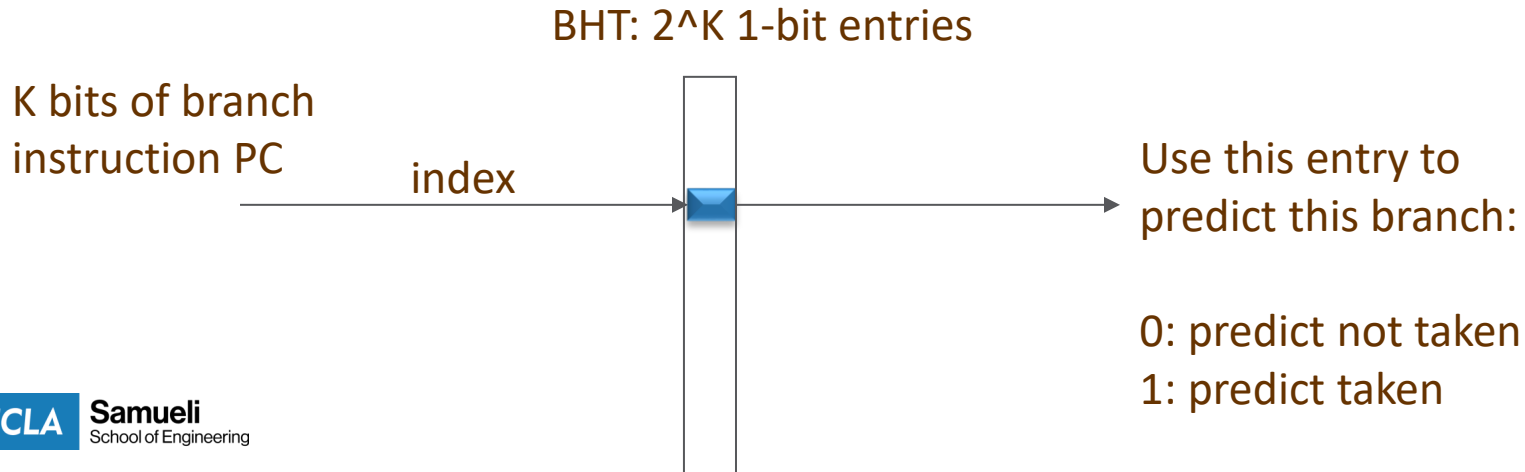
# Control Hazards: Dynamic Prediction

---

- Using a Branch predictor
  - Predict the outcome dynamically using the branch history
  - Recurring branches tend to repeat their pattern (e.g. loops)

# Control Hazards: Dynamic Prediction

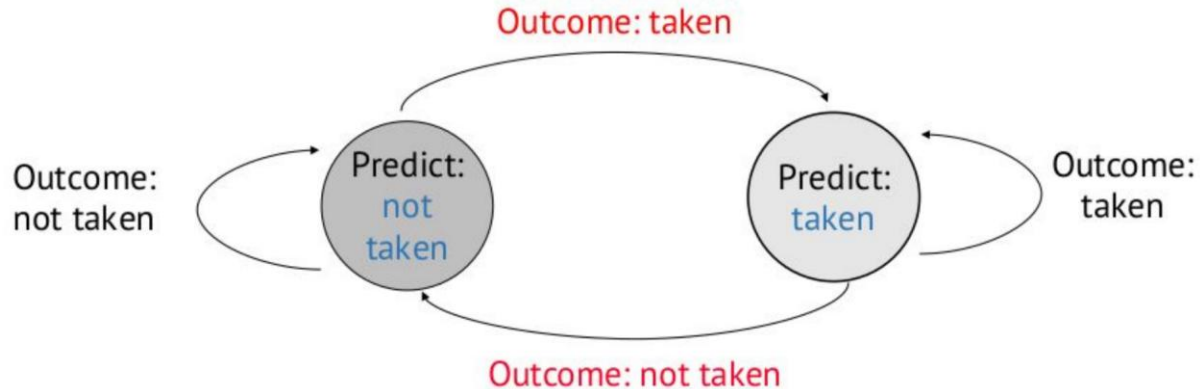
- One-Bit Branch Predictor
  - BTB entry hold a single bit: taken=1 or not\_taken=0.
  - Index the table using lower K bits of PC
  - Update the table when the condition is resolved.



# Control Hazards: Dynamic Prediction

---

- One-Bit Branch Predictor
  - FSM



# Control Hazards: Dynamic Prediction

- One-Bit Branch Predictor Performance

DC08: TTTTTTTTTT ... TTTTTTTTTNTTTTTTTTTT...

← 100,000 iterations →

How often is branch outcome  $\neq$  previous outcome?

2 / 100,000

99.9%

**DC44:** TTTTTT ... TNTTTTTT ... TNTTTTTT ...

2 / 100

98.0%

[illegible]

2 / 2

00.0%

# Whiteboard

---

# Control Hazards: Dynamic Prediction

---

- One-Bit Is not enough!

Example: short loop (8 iterations)

- Taken 7 times, then not taken once
- Actual: TTTTTTTNTTTTTTNTTTTTTNT...
- Pred: XTTTTTT**T**NTTTTT**T**NTTTTT**T**N
- Corr: X\_\_\_\_\_ **MM** \_\_\_\_\_ **MM** \_\_\_\_\_ **MM**
- Misprediction rate:  $2/8 = 25\%$

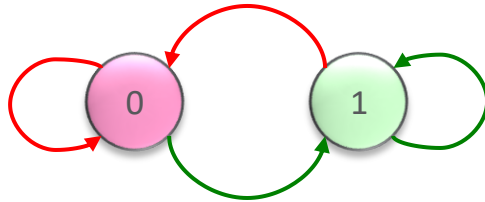
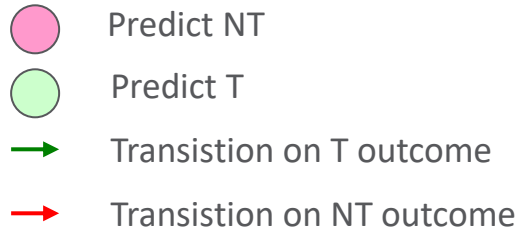
Execute the same loop again

- First always mispredicted (previous outcome was not taken)
- Then 6 predicted correctly
- Then last one mispredicted again

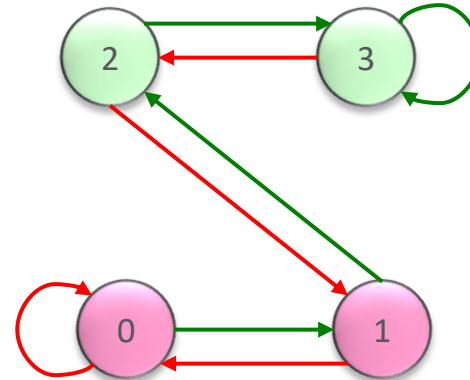
Each fluke/anomaly results in two mispredicts per loop

# Control Hazards: Dynamic Prediction

- Two-Bit Branch Predictor



FSM for Last-time  
Prediction



FSM for 2bC  
(2-bit Counter)

Smith, ISCA 1981

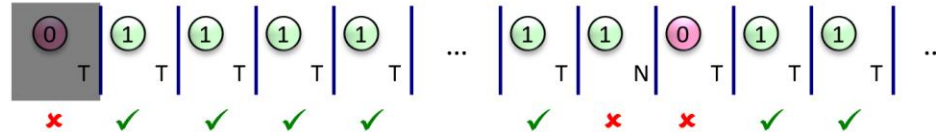


# Control Hazards: Dynamic Prediction

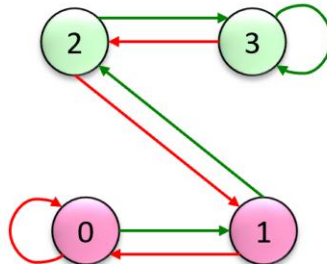
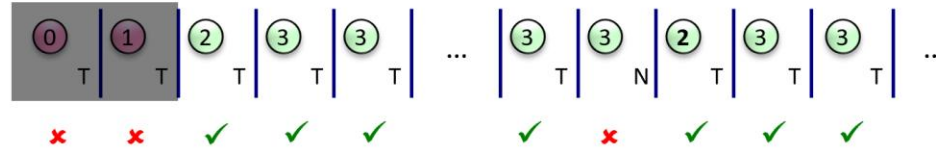
- Two-Bit Branch Predictor

Initial Training/Warm-up

1bC:



2bC:



Only 1 Mispredict per N branches now!

DC08: 99.999%

DC44: 99.0%

DC50: 50.0%

# Control Hazards: Dynamic Prediction

---

- Can we do better?
  - 1-BC and 2-BC predictors use last branch outcome to predict
  - Observation1: local branch correlation
    - The branch outcome is correlated with prior-outcome from same branch
  - Observation2: global branch correlation
    - The branch outcome is correlated with outcome from previous branches

# Control Hazards: Dynamic Prediction

---

## Global branch correlation

```
if (cond1) {  
    --/--  
}  
if (cond1 and cond2) {  
    --/--  
}
```

## Local branch correlation

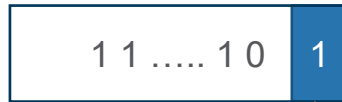
```
for (x: 0 -> 1023) {  
    --/--  
}
```

# Control Hazards: Dynamic Prediction

## Two-level Branch Predictor

- Level1: history of last N branches
- Level2: last direction the branch took

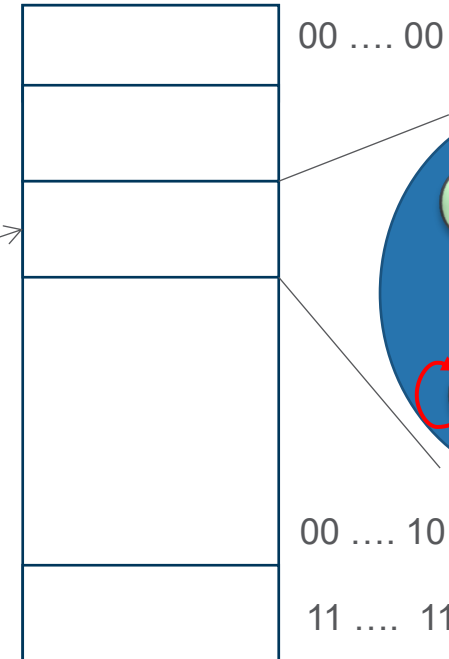
**Level1**  
Branch History Register (BHR)



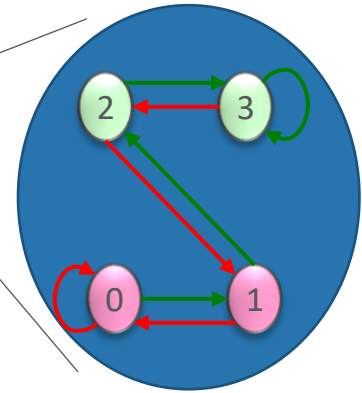
previous one

index

2-Bit BHT



**Level2**



Yeh&patt'92

# Control Hazards: Dynamic Prediction

## Two-level Branch Predictor

Initialization value (0 or 1)

Past

Present



← History length →

1 : branch is taken  
0: branch is not-taken

New BHR = old BHR<<1 | (br\_dir);

### Example:

|                |             |
|----------------|-------------|
|                | BHR 00000   |
| Br1 : taken    | → BHR 00001 |
| Br2: not-taken | → BHR 00010 |
| Br3: taken     | → BHR 00101 |

# Control Hazards: Dynamic Prediction

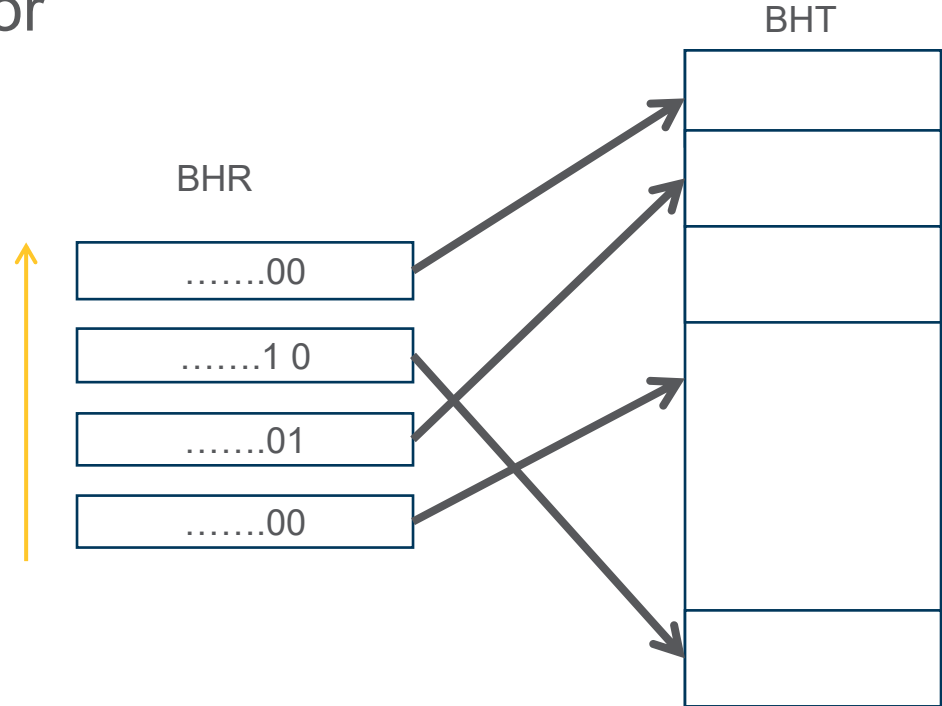
## Two-level Branch Predictor

Branch X: if (cond1)

Branch Y: if (cond 2)

Branch Z : if (cond 1 and cond 2)

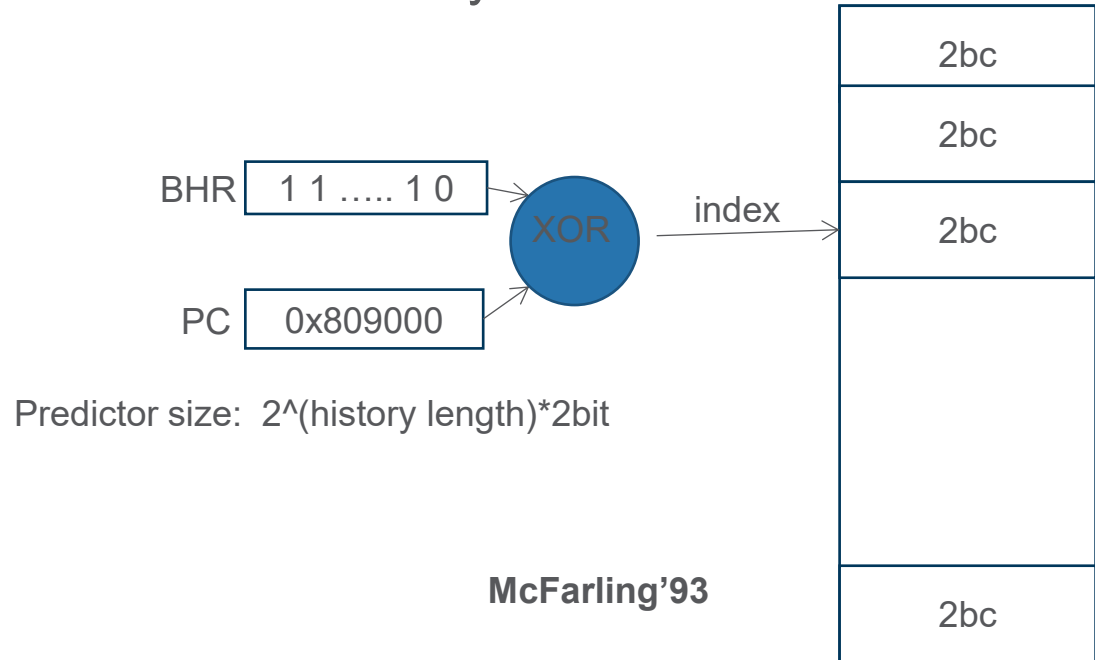
| Branch X | Branch Y | Branch Z |
|----------|----------|----------|
| 1        | 0        | 0        |
| 1        | 1        | 1        |
| 0        | 1        | 0        |
| 0        | 0        | 0        |



# Control Hazards: Dynamic Prediction

## Gshare Branch Predictor

- Different branches may see the same history but have different outcome
- Need additional context



McFarling'93

# Control Hazards: Dynamic Prediction

---

## Gshare Branch Predictor: algorithm

```
predict_func(pc, actual_dir) {  
    index = PC xor BHR  
    taken = (2bit_counters[index] >= 2) ? 1 : 0  
    correctly_predicted = (actual_dir == taken) ? 1 : 0 // stats  
}
```

```
update_func(pc, actual_dir) {  
    index = PC xor BHR  
    if (actual_dir)  
        INC_SAT(2bit_counter[index])  
    else  
        DEC_SAT(2bit_counter[index])  
    BHR = BHR << 1 | actual_dir  
}
```



# Control Hazards: Dynamic Prediction

## Improving Return Call Prediction

- RISC-V JALR
  - $PC = R[rs1] + imm$
- Target depends on rs1

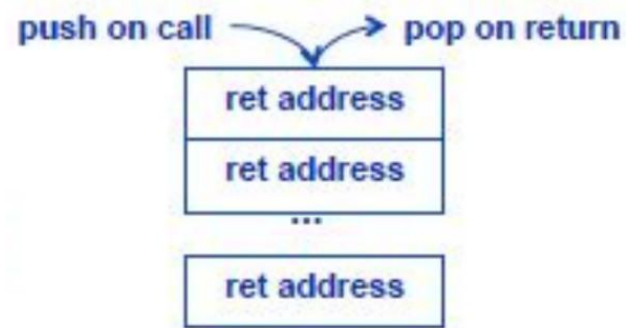
| Control-Flow inst. | Direction at fetch time | Number of possible next fetch addresses? | When is next fetch address resolved? |
|--------------------|-------------------------|------------------------------------------|--------------------------------------|
| Conditional        | Unknown                 | 2                                        | Decode ( $rs1 == rs2$ )              |
| Unconditional      | Always taken            | 1                                        | Decode (PC + offset)                 |
| Call               | Always taken            | 1                                        | Decode (PC + offset)                 |
| Return             | Always taken            | Many                                     | Decode (register dependent)          |
| Indirect           | Always taken            | Many                                     | Decode (register dependent)          |

# Control Hazards: Dynamic Prediction

---

## Return Stack Buffer (RSB)

- Use hardware stack
  - About 4-8 entries
- On function call:
  - Push current return address
- On function return:
  - Pop last return address
  - Use that for prediction

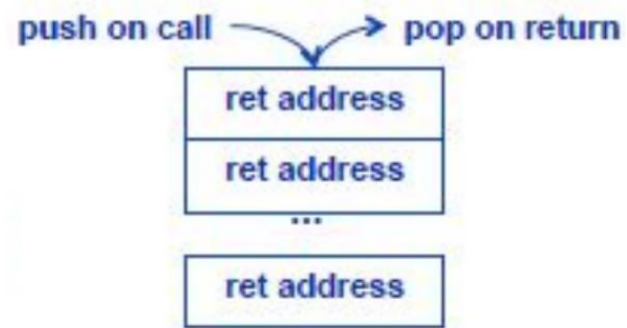


# Control Hazards: Dynamic Prediction

---

## Return Stack Buffer (RSB)

- Integration with BTB:
  - Predict NextPC from BTB or RSB
  - How to choose?
    - Extend BTB with “isRA” column



# Acknowledgement

---

- This course is partly inspired and use materials from the following courses created by my colleagues:
  - ECE-M1126C: Nader Sehatbakhsh @ UCLA
  - CS3220: Hyesoon Kim @ gatech

# Q&A

---